

Helix OTA — Stack Research Note:

SWUpdate

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Evidence-based evaluation of SWUpdate as a candidate device-side update-execution engine for the future Helix Linux phase only. Covers the .swu (cpio) package format, sw-description + Lua parser, C/Lua handlers, double-copy (A/B) via Software collections + bootloader env, streaming installation, suricatta backends (hawkBit DDI, general-purpose HTTP, wfx, Lua suricatta module), delta/zchunk + rdiff, and storage backends. SWUpdate is an embedded-Linux engine with no Android target; it is out of scope for the Android-15-first phase. Assessed here as foundation-corpus input alongside RAUC and OSTree. Overall scorability for the landscape matrix: see §13. A small number of facts depend on edition-specific doc pages and may drift across releases (notably: exact set of streaming-capable handlers, exact crypto digest provider names, and whether wfx is built by default). These are tagged “UNVERIFIED — needs confirmation” inline. Android is not a SWUpdate target — irrelevant to MVP. SWUpdate’s native server protocol is hawkBit DDI; Helix wants a custom Go control plane, so the suricatta[?]server contract must be reimplemented (hawkBit-compatible or general-HTTP) or written as a Lua suricatta module.
Issues	

Field	Value
Issues summary	No fabricated stars/dates/figures; every version/date/claim either cites a consulted URL or is flagged unverified. The GitHub star count was not captured and is intentionally omitted rather than invented.
Fixed	initial research pass (web-verified 2026-06-07)
Fixed summary	Primary sources are the official SWUpdate docs at <code>sbabic.github.io/swupdate</code> (swupdate / suricatta / handlers / delta-update pages) and the upstream <code>github.com/sbabic/swupdate</code> repo (releases, license). Secondary cross-checks (DeepWiki, third-party blogs) are labelled as such. Latest release confirmed 2026.05 (2026-05-29) from the GitHub releases page.
Continuation	Feed into research/ <code>ota_landscape_report.md</code> (the report previously excluded SWUpdate from scoring — this note supplies the missing score) and the engine-selection ADR (ADR-0001). When the Linux phase is scheduled, deepen with a hands-on spike: build a signed <code>.swu</code> , install it double-copy with a bootloader env (U-Boot/EFI Boot Guard), drive it from the Helix Go control plane via either the general-purpose HTTP contract or a Lua suricatta module, and benchmark <code>zchunk delta</code> . Re-verify version/release facts before any decision is locked.

Table of contents

- [§1. Scope & relevance to Helix](#)
- [§2. What SWUpdate is \(snapshot\)](#)
- [§3. The `.swu` package format](#)
- [§4. `sw-description` and the Lua parser](#)
- [§5. Handlers \(C and Lua\) & storage backends](#)
- [§6. A/B, double-copy & bootloader env](#)
- [§7. Streaming installation](#)
- [§8. Suricatta, hawkBit & the control-plane question](#)
- [§9. Delta / incremental updates](#)
- [§10. Security: signing & verification](#)
- [§11. Android fit & Go control-plane fit](#)
- [§12. Wrappability for Helix](#)

- §13. Scorability (1–5, landscape matrix)
- §14. Pros / cons / risks & recommendation
- §15. Sources consulted
- §16. Confidence

§1. Scope & relevance to Helix

Helix OTA is **Android 15 first**: native A/B via Android’s `update_engine / payload.bin`, plus a custom Go control plane. SWUpdate does **not** apply to that phase — it is an embedded-Linux update framework with **no Android target**. Its relevance is strictly the **future Helix Linux / universal phase**, where Helix would need a device-side apply engine for non-Android Linux targets. In that role SWUpdate is one of the three dominant open-source options (alongside RAUC and Mender), and is the most flexible/handler-driven of them: rather than RAUC’s “declarative slots” model, SWUpdate is an extensible install pipeline driven by a per-update manifest (`sw-description`) and pluggable handlers.

This note evaluates SWUpdate against Helix’s locked constraints: a custom **Go** control plane, HTTP delivery (Helix mandates Brotli + HTTP/3→HTTP/2 fallback, REST primary), strong supply-chain integrity, and the ability to **wrap** the engine behind an OS-adapter interface rather than adopt a vendor server stack. SWUpdate is attractive here because its device daemon (`suricata`) is explicitly designed to talk to a remote server, and the server side is replaceable.

§2. What SWUpdate is (snapshot)

Attribute	Value	Source confidence
Project	SWUpdate (“software update for embedded systems”)	high — official docs
Origin / maintainer	Created/maintained by Stefano Babic (<code>sbabic</code>); widely used in Yocto/OE-based products	high
Repo	<code>github.com/sbabic/swupdate</code>	high
License	GPL-2.0-only (SPDX <code>GPL-2.0-only</code>) per GitHub	high — GitHub repo metadata
Language	Primarily C , with Lua for extensibility (parser, handlers, <code>suricata</code> modules)	high — docs
Latest release	2026.05, released 2026-05-29 (per GitHub releases at time of fetch); prior: 2025.12 (2025-12-03), 2025.05 (2025-05-06), 2024.12.1	medium — single fetch, re-verify before locking

Attribute	Value	Source confidence
	(2026-01-22), 2024.12 (2024-12-01)	
Release cadence	Roughly twice-yearly date-versioned releases (YYYY.MM)	medium — inferred from tag pattern
GitHub stars	Not captured — intentionally omitted (anti-bluff; do not invent)	n/a
Default web/server libs	Mongoose web server (embedded HTTP), upgraded to 7.21 in 2026.05; libcurl for client transport	medium — release notes

SWUpdate runs on the target as either a one-shot installer (`swupdate -i image.swu`), an embedded web-server mode (Mongoose, local upload UI/API), or a **suricata** polling daemon. It does **not** ship a fleet control plane — the server side is the integrator’s responsibility (exactly what Helix wants).

§3. The .swu package format

An update is delivered as a single **.swu** file, which is a **cpio archive**:

- cpio was chosen because it is “a simple, well-established, and streamable format.”
- Supported variants: **New ASCII format (magic 070701)** and **New CRC format (magic 070702)**. The CRC variant carries a 32-bit checksum of data bytes that SWUpdate verifies.
- **sw-description must be the first entry** in the archive, followed by the artifact files it references.
- cpio’s per-file size is bounded by its **32-bit** size field (≈ 4 GB per single artifact) — relevant only for very large monolithic images.

Because cpio is streamable and **sw-description** comes first, SWUpdate can parse the manifest and begin installing the first artifact while later artifacts are still arriving over the network — valuable on low-RAM / low-flash devices.

Source: SWUpdate docs `swupdate.html`; DeepWiki usage guide (secondary).

§4. sw-description and the Lua parser

sw-description is the per-update manifest: it lists images/files, target devices/volumes, handlers, versions, hashes, and install options. Two parsers are supported:

- **Internal parser** based on **libconfig** (the default, libconfig syntax).
- **External Lua parser** — **sw-description** format can be customized by calling an external parser written in **Lua**.

This is a notable flexibility advantage over RAUC's fixed manifest: Helix could, in the Linux phase, shape the manifest contract to match its control-plane metadata without forking the engine.

Source: SWUpdate docs `swupdate.html`.

§5. Handlers (C and Lua) & storage backends

SWUpdate's install model is **handler-driven**: each artifact in `sw-description` names a handler responsible for writing it. Handlers may be **built-in C** or **Lua scripts** loaded at runtime.

Documented install targets / storage backends include:

- **Raw block / embedded media**: eMMC, SD, Raw NAND, NOR and SPI-NOR flashes (raw handler).
- **UBI volumes** (NAND/UBI handler).
- **Single-file update inside a mounted filesystem**.
- **Partitioner / diskpart** support; **btrfs** partitioner support added in 2023.05 (per release notes, secondary).
- **Delta handlers** (rdiff, zchunk — see §9), which assemble the target artifact then pass it to a **secondary handler** to perform the actual write.

The handler abstraction means a target-specific or Helix-specific write path can be added without touching the core. UNVERIFIED — needs confirmation: the exact, current full handler list and which handlers support streaming should be re-read from `handlers.html` for the targeted release.

Source: SWUpdate docs `swupdate.html`, `handlers.html`.

§6. A/B, double-copy & bootloader env

SWUpdate does **not** impose a slot model. Two approaches are documented:

- **Single-copy**: SWUpdate runs in an **initrd**, updating the (single) main system in place. Atomicity/rollback then relies on the `initrd` flow rather than a redundant slot.
- **Double-copy (A/B)**: achieved via **Software collections** — the manifest defines multiple collections (e.g. `stable`, `alt`) and the operator selects the target with `--select <collection>, <mode>` (e.g. `--select stable, alt`), so the update is written to the **inactive** copy. This is SWUpdate's A/B equivalent, but it is **convention + configuration**, not a built-in slot manager like RAUC or `update_engine`.

Atomic switchover and rollback are delegated to the **bootloader environment**, which SWUpdate can set/erase:

- **U-Boot** environment variables.
- **GRUB** environment block variables.
- **EFI Boot Guard** variables.

So the typical A/B flow is: write inactive copy → set bootloader “try new copy / boot count” variable → reboot → confirm-good (clear the try flag) or auto-revert on failed boot. Compared to RAUC, the slot/rollback logic is **less turnkey and more integrator-assembled**.

Source: SWUpdate docs `swupdate.html`.

§7. Streaming installation

A core SWUpdate design principle: “SWUpdate is thought to be able to **stream the received image directly into the target, without any temporary copy**.” Streaming is **configurable per-image** — some artifacts can stream while others use a temporary extraction. The rationale given is that on resource-constrained systems “the amount of RAM to copy the images could be not enough.”

This pairs well with the `cpio` format (§3) and is a genuine differentiator for small devices. For Helix’s mandated HTTP delivery, SWUpdate’s client uses `libcurl`, so Brotli content-encoding and HTTP/2 are feasible at the transport layer (HTTP/3 support would depend on the `libcurl` build — UNVERIFIED — needs confirmation).

Source: SWUpdate docs `swupdate.html`.

§8. Suricatta, hawkBit & the control-plane question

Suricatta is SWUpdate’s polling daemon mode (named after the meerkat, a mongoose relative): it “regularly polls a remote server for updates, downloads, and installs them.” Suricatta supports multiple backends, **selectable at runtime**:

“If multiple server support is compiled in, the `-S / --server` option or a `server` entry in the configuration file’s `[suricatta]` section selects the one to use at run-time.”

Backends:

1. **hawkBit** — via the **hawkBit Direct Device Integration (DDI) API**. This is SWUpdate’s most mature/native server protocol and pairs with Eclipse hawkBit (already covered in `eclipse-hawkbit.md`).
2. **General-purpose HTTP server** — “a very simple backend that uses standard HTTP response codes to signal if an update is available.” The device sends device info via **GET query parameters**; the server replies with status codes:

Code	Meaning
302 Found	New software available at URL in the <code>Location</code> header
400 Bad Request	Query parameters missing / wrong format
403 Forbidden	Client certificate not valid
404 Not Found	No update available for this device

Code	Meaning
503 Unavailable	Update available but server busy (with Retry-after)

Responses can carry Content-MD5 and Retry-after. A mock reference server lives at `examples/suricatta/server_general.py`.

3. **wfx** — integration with the Eclipse **wfx** workflow executor (Device Artifact Update workflows). UNVERIFIED — needs confirmation whether wfx support is built by default vs. opt-in.
4. **Lua suricatta module** — `server_lua.c` bridges the suricatta interface into Lua, so a **complete suricatta backend can be written in Lua** instead of C (“an option for writing such in C instead of Lua”).

Control-plane implication for Helix: there are three viable integration paths for the custom Go control plane, in increasing effort/decreasing coupling: - (a) Make the Go control plane speak the **general-purpose HTTP contract** above — trivial to implement in Go (it is just status codes + a Location URL + query params). - (b) Make the Go control plane implement the **hawkBit DDI** API surface (more work, but reuses SWUpdate’s most-tested client path and aligns with the existing hawkBit research). - (c) Ship a **Lua suricatta module** on-device that speaks Helix’s own REST/Brotli/HTTP-3 protocol directly — maximum protocol freedom, on-device Lua maintenance cost.

Option (a) is the cheapest and most aligned with “minimal wrapping + custom Go control plane.” Option (c) is the most flexible and lets Helix keep its mandated transport semantics end-to-end.

Source: SWUpdate docs `suricatta.html`; `suricatta/server_general.c` (master).

§9. Delta / incremental updates

SWUpdate supports two delta mechanisms:

- **zchunk** — the chosen format for delta delivery. A zchunk file has a header describing all chunks; the device generates the header of the **running** artifact, compares, and downloads **only the missing chunks**, reusing the rest locally. The delta handler assembles the full artifact then hands off to a **secondary handler** for the write.
- **rdiff** (librsync) — applies binary delta patches generated by librsync’s `rdiff`. Documented prime use case: read-only rootfs receiving small security/feature patches.

2026.05 release notes mention “**Delta Update with Hawkbit**,” indicating server-coordinated delta is an active, maintained path.

For Helix this matters because the design corpus mandates delta/incremental updates (cf. ADR-0005). SWUpdate’s zchunk model is range-request friendly (download missing chunks) and fits HTTP range/CDN delivery well.

Source: SWUpdate docs `delta-update.html`, `handlers.html`; 2026.05 release notes.

§10. Security: signing & verification

- **“Images are authenticated and verified before installing.”**
- Configurable digest/crypto providers — documented options include **OpenSSL CMS (PKCS#7)** and **OpenSSL RSA** signing, with CA-path certificate validation. (Exact current provider identifiers — e.g. `opensslCMS` / `opensslRSA` — should be re-confirmed per release: UNVERIFIED — needs confirmation.)
- The `.swu` (`cpio`) container is signed/verified; per-artifact hashes are declared in `sw-description`.
- Encryption of artifacts is also supported (symmetric, configured in the manifest) — UNVERIFIED — needs confirmation of exact cipher/mode for the targeted release.

This is sufficient for Helix’s integrity needs at the device-apply layer, though Helix’s broader supply-chain trust model (TUF/Uptane per ADR-0002) would sit **above** SWUpdate, not inside it.

Source: SWUpdate docs `swupdate.html`.

§11. Android fit & Go control-plane fit

- **Android-15 fit: none.** SWUpdate is an embedded-Linux engine; it does not run as the Android updater, does not produce/consume `payload.bin`, and does not integrate with Android’s `update_engine` / `boot_control` HAL / Virtual A/B. For the Android MVP it is irrelevant. (Scores 1 for Android-15 fit in §13.)
- **Go fit: pattern-level, good on the server contract.** SWUpdate itself is C/Lua, so there is no Go device-side reuse. But its **server contracts are easy Go targets** — the general-purpose HTTP backend is just status codes + headers, trivially served by a Go HTTP handler; hawkBit DDI is a documented REST API a Go service can implement. Helix’s Go control plane does **not** need to embed any SWUpdate code to drive a SWUpdate fleet. (Scores 3 — usable, integration work on the protocol, no Go reuse of the engine.)

§12. Wrappability for Helix

“Wrapping” in Helix means putting the engine behind an OS-adaptor interface and driving it from the Go control plane, without adopting the engine’s vendor server.

- SWUpdate is **highly wrappable** as a **device-side apply engine** for Linux: it can be invoked one-shot (`swupdate -i`), or run as `suricata` against a Helix-supplied server, and its handler/parser/`suricata` layers are all replaceable in Lua. The server side is explicitly the integrator’s.
- The caveat vs. RAUC: A/B slot management and rollback are **assembled by the integrator** (Software collections + bootloader env), so the wrapper carries more responsibility for the slot/confirm/rollback state machine than with RAUC’s built-in slots.

- Net: strong wrappability for the **Linux phase only**; **zero** wrappability for Android (it doesn't run there). Wrap-ability scored **4** for the Linux role (one point below RAUC because the A/B/rollback orchestration is less turnkey).

§13. Scorability (1–5, landscape matrix)

Scored on the existing `ota_landscape_report.md` rubric (§2.1): **1–5, relative to Helix's locked strategy** (Android-15-first, native A/B, custom Go control plane, minimal wrapping). Higher = better fit for Helix. The report previously **excluded** SWUpdate from scoring; this row fills that gap. As with RAUC/OSTree, a low A/B/Android score reflects the **Linux-phase-only** role and is not disqualifying for that role.

Stack (slug)	Mechanism	Rollback	Staged-rollout	A/B	Android-15 fit	Go fit	Wrap-ability	License	Maturity
swupdate	4	3	3	3	1	3	4	4	4

Per-criterion justification (traceable to sections above):

- **Mechanism (4):** Handler-driven streaming install pipeline; `cpio .swu + sw-` description; very flexible but not Helix's native Android mechanism. [§3–§5, §7]
- **Rollback (3):** Rollback is **delegated to the bootloader env** (U-Boot/GRUB/EFI Boot Guard) and assembled by the integrator; no built-in slot-confirm state machine like RAUC/`update_engine`. [§6]
- **Staged-rollout (3):** No native rollout orchestration in the engine, but **suricata + hawkBit DDI/wfx** gives a real path to staged rollout when paired with a server (better than RAUC's 1, worse than hawkBit's 5 since the engine itself does none). [§8]
- **A/B (3):** Double-copy is achievable via **Software collections + bootloader env**, but it is convention/config, not a first-class slot manager. Below RAUC's 5. [§6]
- **Android-15 fit (1):** No Android target at all; irrelevant to the MVP. [§1, §11]
- **Go fit (3):** No Go reuse of the engine, but the **server contract is a trivial Go target** (general-HTTP) or a documented one (hawkBit DDI). [§8, §11]
- **Wrap-ability (4):** Excellent as a wrapped Linux apply engine (one-shot or `suricata`; Lua-extensible; integrator owns the server); minus one vs RAUC because A/B/rollback orchestration is less turnkey. Zero for Android. [§12]
- **License (4): GPL-2.0-only** — copyleft. Fine for a separate on-device daemon driven over a protocol (no linking into Helix proprietary code), but GPL obligations on any modified/distributed SWUpdate binaries must be respected; matches RAUC's 4-tier (RAUC is LGPL, slightly more permissive — hence not a 5 here). [§2]
- **Maturity (4):** Long-lived, actively maintained (2026.05 release 2026-05-29), broad Yocto/OE adoption, hawkBit/wfx/delta features maintained. Not a 5 only because Android-phase relevance is nil and production scale evidence wasn't independently quantified here. [§2]

Total 29/45 — comparable to RAUC (30) and above OSTree (25) in the existing matrix, i.e. a **credible Linux-phase device-apply candidate**, ranked just below RAUC primarily on turnkey A/B/rollback, and well below the Android-phase leaders (`aosp-update-engine` 39, `eclipse-hawkbit` 36) — expected, since SWUpdate is out of scope for the Android MVP.

§14. Pros / cons / risks & recommendation

Pros - Streaming install with minimal scratch storage — strong on constrained devices. [§7] - Extremely flexible: Lua parser, C/Lua handlers, Lua suricatta module — shapeable to Helix's metadata/protocol without forking. [§4, §5, §8] - Server side is the integrator's; **three** clean integration paths for a Go control plane (general-HTTP, hawkBit DDI, Lua module). [§8] - Real delta support (zchunk + rdiff) aligning with ADR-0005. [§9] - Broad bootloader env support (U-Boot/GRUB/EFI Boot Guard). [§6] - Mature, actively released (2026.05). [§2]

Cons / risks - **No Android target** — irrelevant to the Android-15-first MVP. [§1, §11] - A/B and rollback are **integrator-assembled** (Software collections + bootloader env), more work and more failure surface than RAUC's built-in slots. [§6] - **GPL-2.0-only** copyleft — manageable for a separate daemon, but distribution of modified binaries carries obligations; review before shipping device images. [§2] - Native server protocol is hawkBit DDI; to use Helix's mandated REST/Brotli/HTTP-3 transport semantics end-to-end you must either map onto the general-HTTP contract or write a Lua suricatta module. [§8]

Recommendation: Keep SWUpdate as a **strong, optional wrappable device-apply engine for the future Helix Linux phase**, on par with RAUC and ahead of OSTree for this role. **Do not adopt for the Android MVP** (no Android support). When the Linux phase is scheduled, run a spike comparing SWUpdate (Software collections + bootloader env) vs RAUC (built-in slots) for A/B turnkey-ness, and prototype the Helix Go control plane against SWUpdate's general-purpose HTTP contract first (cheapest), with a Lua suricatta module as the fallback for full transport fidelity.

§15. Sources consulted

Primary (official): - SWUpdate docs — main: <https://sbabic.github.io/swupdate/swupdate.html> - SWUpdate docs — suricatta: <https://sbabic.github.io/swupdate/suricatta.html> - SWUpdate docs — handlers: <https://sbabic.github.io/swupdate/handlers.html> - SWUpdate docs — delta update: <https://sbabic.github.io/swupdate/delta-update.html> - SWUpdate docs — index: <https://sbabic.github.io/swupdate/> - GitHub — releases (latest 2026.05, 2026-05-29): <https://github.com/sbabik/swupdate/releases> - GitHub — suricatta/server_general.c (general-HTTP backend): https://github.com/sbabik/swupdate/blob/master/suricatta/server_general.c - GitHub — doc/source/suricatta.rst: <https://github.com/sbabik/swupdate/blob/master/doc/source/suricatta.rst>

Secondary (cross-check, labelled): - DeepWiki SWUpdate usage guides: <https://deepwiki.com/sbabik/swupdate/4-usage-guides> - The Good Penguin — Delta OTA Update with SWUpdate: <https://www.thegoodpenguin.co.uk/blog/delta-ota-update-with-swupdate/>

Internal cross-references: - research/ota_landscape_report.md (§2.1 rubric; SWUpdate previously excluded from scoring — this note supplies the score) - stacks/rauc.md, stacks/ostree.md, stacks/eclipse-hawkbit.md, stacks/mender.md - research/adr/adr-0001-wrapped-engine.md, adr-0002-supply-chain-trust.md, adr-0005-delta-updates.md

§16. Confidence

Overall confidence: HIGH on the architecture/feature claims (package format, suricata backends incl. Lua module + runtime selection, double-copy/bootloader env, streaming, delta zchunk/rdiff, GPL-2.0-only license, 2026.05 release) — all traced to official docs / GitHub fetched 2026-06-07. **MEDIUM/LOW** and tagged **UNVERIFIED** — **needs confirmation** on: exact current handler list and which stream, precise crypto provider identifiers and encryption cipher/mode, whether wfx is built by default, and HTTP/3-via-libcurl availability. GitHub star count was deliberately **not** asserted. Re-verify all version/release facts before any decision is locked in ADR-0001.